



[Click to Take the FREE Deep Learning Performance Crash-Course](#)



A Gentle Introduction to Dropout for Regularizing Deep Neural Networks

by **Jason Brownlee** on [December 3, 2018](#) in **Deep Learning Performance**

Tweet

Share

Share

Last Updated on August 6, 2019

Deep learning neural networks are likely to quickly overfit a training dataset with few examples.

Ensembles of neural networks with different model configurations are known to reduce overfitting, but require the additional computational expense of training and maintaining multiple models.

A single model can be used to simulate having a large number of different network architectures by randomly dropping out nodes during training. This is called dropout and offers a very computationally cheap and remarkably effective regularization method to [reduce overfitting and improve generalization error](#) in deep neural networks of all kinds.

In this post, you will discover the use of dropout regularization for reducing overfitting and improving the generalization of deep neural networks.

After reading this post, you will know:

- Large weights in a neural network are a sign of a more complex network that has overfit the training data.
- Probabilistically dropping out nodes in the network is a simple and effective regularization method.
- A large network with more training and the use of a weight constraint are suggested when using dropout.

Kick-start your project with my new book [Better Deep Learning](#), including *step-by-step tutorials* and the *Python source code* files for all examples.

Let's get started.

Start Machine Learning



You can master applied Machine Learning **without math or fancy degrees**.

Find out how in this *free* and *practical* course.

START MY EMAIL COURSE



A Gentle Introduction to Dropout for Regularizing Deep Neural Networks
Photo by [Jocelyn Kinghorn](#), some rights reserved.

Overview

This tutorial is divided into five parts; they are:

1. Problem With Overfitting
2. Randomly Drop Nodes
3. How to Dropout
4. Examples of Using Dropout
5. Tips for Using Dropout Regularization

Problem With Overfitting

Large neural nets trained on relatively small datasets can overfit.

This has the effect of the model learning the statistical noise in the training data, which results in poor performance when the model is evaluated on new data, e.g. a test dataset. Generalization error increases due to overfitting.

One approach to reduce overfitting is to fit all possible different neural networks on the same dataset and to average the predictions from each model. This is not feasible for a large collection of different models, called an ensemble.



With unlimited computation, the best way to “regularize” is to average over all possible settings of the parameters, weighting each equally by its performance on the training data.

— Dropout: A Simple Way to Prevent Neural Networks from C

Start Machine Learning

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees.** Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

A problem even with the ensemble approximation is that it requires multiple models to be fit and stored, which can be a challenge if the models are large, requiring days or weeks to train and tune.

Randomly Drop Nodes

Dropout is a regularization method that approximates training a large number of neural networks with different architectures in parallel.

During training, some number of layer outputs are randomly ignored or “*dropped out*.” This has the effect of making the layer look-like and be treated-like a layer with a different number of nodes and connectivity to the prior layer. In effect, each update to a layer during training is performed with a different “*view*” of the configured layer.

“ *By dropping a unit out, we mean temporarily removing it from the network, along with all its incoming and outgoing connections*

— [Dropout: A Simple Way to Prevent Neural Networks from Overfitting](#), 2014.

Dropout has the effect of making the training process noisy, forcing nodes within a layer to probabilistically take on more or less responsibility for the inputs.

This conceptualization suggests that perhaps dropout breaks-up situations where network layers co-adapt to correct mistakes from prior layers, in turn making the model more robust.

“ *... units may change in a way that they fix up the mistakes of the other units. This may lead to complex co-adaptations. This in turn leads to overfitting because these co-adaptations do not generalize to unseen data. [...]*

— [Dropout: A Simple Way to Prevent Neural Networks from Overfitting](#), 2014.

Dropout simulates a sparse activation from a given layer, which interestingly, in turn, encourages the network to actually learn a sparse representation as a side-effect. As such, it may be used as an alternative to activity regularization for encouraging sparse representations in autoencoder models.

“ *We found that as a side-effect of doing dropout, the activations of the hidden units become sparse, even when no sparsity inducing regularizers are present.*

Start Machine Learning

— [Dropout: A Simple Way to Prevent Neural Networks from Overfitting](#), 2014.

Because the outputs of a layer under dropout are randomly subsampled, it has the effect of reducing the capacity or thinning the network during training. As such, a wider network can be trained with dropout.

Start Machine Learning



You can master applied Machine Learning **without math or fancy degrees.**
Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE

Want Better Results with

Take my free 7-day email crash course

Click to sign-up and also get a free PDF

How to Dropout

Dropout is implemented per-layer in a neural network.

It can be used with most types of layers, such as dense fully connected layers, convolutional layers, and recurrent layers such as the long short-term memory network layer.

Dropout may be implemented on any or all hidden layers in the network as well as the visible or input layer. It is not used on the output layer.

“The term “dropout” refers to dropping out units (hidden and visible) in a neural network.

— Dropout: A Simple Way to Prevent Neural Networks from Overfitting, 2014.

A new hyperparameter is introduced that specifies the probability at which outputs of the layer are dropped out, or inversely, the probability at which outputs of the layer are retained. The interpretation is an implementation detail that can differ from paper to code library.

A common value is a probability of 0.5 for retaining the output of each node in a hidden layer and a value close to 1.0, such as 0.8, for retaining inputs from the visible layer.

“In the simplest case, each unit is retained with a fixed probability p independent of other units, where p can be chosen using a validation set or can simply be set at 0.5, which seems to be close to optimal for a wide range of networks and tasks. For the input units, however, the optimal probability of retention is usually closer to 1 than to 0.5.

— Dropout: A Simple Way to Prevent Neural Networks from Overfitting, 2014.

Dropout is not used after training when making a prediction with the fit network.

The weights of the network will be larger than normal because the weights are first scaled by the chosen dropout rate. The network is used to make predictions.

“If a unit is retained with probability p during training, the outgoing weights of that unit are multiplied by p at test time

— Dropout: A Simple Way to Prevent Neural Networks from Overfitting, 2014.

The rescaling of the weights can be performed at training time or at test time. This is sometimes called “inverse dropout” and done at training. Both the Keras and PyTorch deep learning libraries implement it.

“At test time, we scale down the output by the dropout rate implemented by doing both operations at training time

Start Machine Learning

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

which is often the way it's implemented in practice

— Page 109, [Deep Learning With Python](#), 2017.

Dropout works well in practice, perhaps replacing the need for weight regularization (e.g. weight decay) and [activity regularization](#) (e.g. representation sparsity).

“... dropout is more effective than other standard computationally inexpensive regularizers, such as weight decay, filter norm constraints and sparse activity regularization. Dropout may also be combined with other forms of regularization to yield a further improvement.

— Page 265, [Deep Learning](#), 2016.

Examples of using Dropout

This section summarizes some examples where dropout was used in recent research papers to provide a suggestion for how and where it may be used.

Geoffrey Hinton, et al. in their 2012 paper that first introduced dropout titled “[Improving neural networks by preventing co-adaptation of feature detectors](#)” applied used the method with a range of different neural networks on different problem types achieving improved results, including handwritten digit recognition ([MNIST](#)), photo classification (CIFAR-10), and speech recognition (TIMIT).

“... we use the same dropout rates – 50% dropout for all hidden units and 20% dropout for visible units

Nitish Srivastava, et al. in their 2014 journal paper introducing dropout titled “[Dropout: A Simple Way to Prevent Neural Networks from Overfitting](#)” used dropout on a wide range of computer vision, speech recognition, and text classification tasks and found that it consistently improved performance on each problem.

“We trained dropout neural networks for classification problems on data sets in different domains. We found that dropout improved generalization performance on all data sets compared to neural networks that did not use dropout.

On the computer vision problems, different dropout rates were used in conjunction with a max-norm weight constraint.

“Dropout was applied to all the layers of the network with the probability of retaining the unit being $p = (0.9, 0.75, 0.75, 0.5, 0.5, 0.5)$ for the different layers of the network (going from input to convolutional layers to fully connected layers). In addition, the max-norm weight constraint was used on all the layers. [...]

A simpler configuration was used for the text classification task.

“We used probability of retention $p = 0.8$ in the input layer and a max-norm weight constraint with $c = 4$ was used in all the layers.

Alex Krizhevsky, et al. in their famous 2012 paper titled “[ImageNet Classification with Deep Convolutional Neural Networks](#)” achieved (at the time) state-of-the-art results for ph

Start Machine Learning

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE

il
ep

convolutional neural networks and dropout regularization.

“ We use dropout in the first two fully-connected layers [of the model]. Without dropout, our network exhibits substantial overfitting. Dropout roughly doubles the number of iterations required to converge.

George Dahl, et al. in their 2013 paper titled “[Improving deep neural networks for LVCSR using rectified linear units and dropout](#)” used a deep neural network with rectified linear activation functions and dropout to achieve (at the time) state-of-the-art results on a standard speech recognition task. They used a bayesian optimization procedure to configure the choice of activation function and the amount of dropout.

“ ... the Bayesian optimization procedure learned that dropout wasn’t helpful for sigmoid nets of the sizes we trained. In general, ReLUs and dropout seem to work quite well together.

Tips for Using Dropout Regularization

This section provides some tips for using dropout regularization with your neural network.

Use With All Network Types

Dropout regularization is a generic approach.

It can be used with most, perhaps all, types of neural network models, not least the most common network types of Multilayer Perceptrons, Convolutional Neural Networks, and Long Short-Term Memory Recurrent Neural Networks.

In the case of LSTMs, it may be desirable to use different dropout rates for the input and recurrent connections.

Dropout Rate

The default interpretation of the dropout hyperparameter is the probability of training a given node in a layer, where 1.0 means no dropout, and 0.0 means no outputs from the layer.

A good value for dropout in a hidden layer is between 0.5 and 0.8. Input layers use a larger dropout rate, such as of 0.8.

Use a Larger Network

It is common for larger networks (more layers or more nodes)

When using dropout regularization, it is possible to use larger networks with less risk of overfitting. In fact, a large network (more nodes per layer) may be required as dropout will probabilistically reduce the capacity of the network.

A good rule of thumb is to divide the number of nodes in the large network by the dropout rate to get the number of nodes to use that as the number of nodes in the new network that uses dropout. For example, a network with 100 nodes and a proposed dropout rate of 0.5 will require 200 nodes ($100 / 0.5$).

“ If n is the number of hidden units in any layer and p is the dropout rate, the dropout net should have at least n/p units

— [Dropout: A Simple Way to Prevent Neural Networks from Overfitting](#)

Grid Search Parameters

Start Machine Learning

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees.**

Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

Rather than guess at a suitable dropout rate for your network, test different rates systematically.

For example, test values between 1.0 and 0.1 in increments of 0.1.

This will both help you discover what works best for your specific model and dataset, as well as how sensitive the model is to the dropout rate. A more sensitive model may be unstable and could benefit from an increase in size.

Use a Weight Constraint

Network weights will increase in size in response to the probabilistic removal of layer activations.

Large weight size can be a sign of an unstable network.

To counter this effect a weight constraint can be imposed to force the norm (magnitude) of all weights in a layer to be below a specified value. For example, the maximum norm constraint is recommended with a value between 3-4.

“ [...] we can use max-norm regularization. This constrains the norm of the vector of incoming weights at each hidden unit to be bound by a constant c . Typical values of c range from 3 to 4.

— Dropout: A Simple Way to Prevent Neural Networks from Overfitting, 2014.

This does introduce an additional hyperparameter that may require tuning for the model.

Use With Smaller Datasets

Like other regularization methods, dropout is more effective on those problems where there is a limited amount of training data and the model is likely to overfit the training data.

Problems where there is a large amount of training data may see less benefit from using dropout.

“ For very large datasets, regularization confers little reduction in generalization error. In these cases, the computational cost of using dropout and larger models may outweigh the benefit of regularization.

— Page 265, [Deep Learning](#), 2016.

Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Books

- Section 7.12 Dropout, [Deep Learning](#), 2016.
- Section 4.4.3 Adding dropout, [Deep Learning With Python](#)

Papers

- [Improving neural networks by preventing co-adaptation of feature groups](#)
- [Dropout: A Simple Way to Prevent Neural Networks from Overfitting](#)
- [Improving deep neural networks for LVCSR using rectified linear units](#)
- [Dropout Training as Adaptive Regularization](#), 2013.

Posts

Start Machine Learning

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE

- Dropout Regularization in Deep Learning Models With Keras
- How to Use Dropout with LSTM Networks for Time Series Forecasting

Articles

- Dropout (neural networks), Wikipedia.
- Regularization, CS231n Convolutional Neural Networks for Visual Recognition
- How was 'Dropout' conceived? Was there an 'aha' moment?

Summary

In this post, you discovered the use of dropout regularization for reducing overfitting and improving the generalization of deep neural networks.

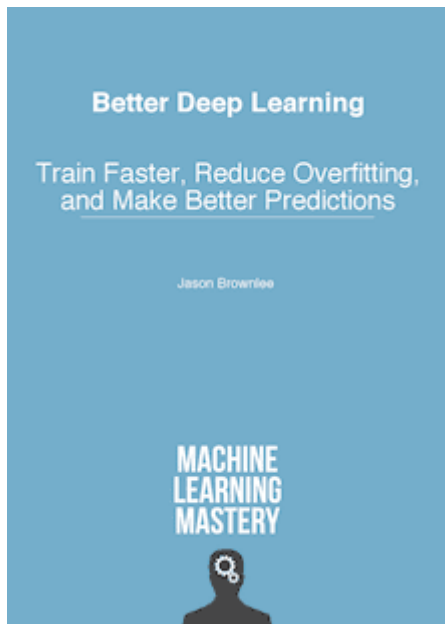
Specifically, you learned:

- Large weights in a neural network are a sign of a more complex network that has overfit the training data.
- Probabilistically dropping out nodes in the network is a simple and effective regularization method.
- A large network with more training and the use of a weight constraint are suggested when using dropout.

Do you have any questions?

Ask your questions in the comments below and I will do my best to answer.

Develop Better Deep Learning Models Today!



Train Faster, Reduce Overfitting, and Ensembles

...with just a few lines of python code

Discover how in my new Ebook:

[Better Deep Learning](#)

It provides **self-study tutorials** on topics like:

weight decay, batch normalization, dropout, model stacking and much more...

Bring better deep learning to your projects!

Start

Start Machine Learning

SEE WHAT'S INSIDE

Tweet

Share

Share



About Jason Brownlee

Jason Brownlee, PhD is a machine learning specialist who teaches you new machine learning methods via hands-on tutorials.

[View all posts by Jason Brownlee →](#)

Start Machine Learning



You can master applied Machine Learning **without math or fancy degrees.**

Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE

34 Responses to *A Gentle Introduction to Dropout for Regularizing Deep Neural Networks*



Shane December 3, 2018 at 12:46 pm #

REPLY ↩

This is off-topic.

Why do you write most blogs on deep learning methods instead of other methods more suitable for time series data?



Jason Brownlee December 3, 2018 at 2:36 pm #

REPLY ↩

Good question, generally because I get 100:1 more questions and interest in deep learning, and specifically deep learning with python open source libraries.



Shane December 8, 2018 at 4:13 am #

REPLY ↩

So if you are working on a personal project, will you use deep learning or the method that gives best results?



Jason Brownlee December 8, 2018 at 7:13 am #

REPLY ↩

I use the method that gives the best results and the lowest complexity for a project.



Claire December 12, 2018 at 8:53 pm #

REPLY ↩

Thanks for sharing. A really easy to understand explanation – I look forward to putting it into action in my next project



Jason Brownlee December 13, 2018 at 7:52 am #

Thanks.



David December 31, 2018 at 11:41 pm #

Great reading to finish my 2018. Happy new year and

Start Machine Learning

Start Machine Learning

You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

START MY EMAIL COURSE



Jason Brownlee January 1, 2019 at 6:15 am #

REPLY ↩

Thanks.



Erestu May 7, 2019 at 1:01 pm #

REPLY ↩

Thank you for writing this introducton.It was so friendly for a new DL learner.Really easy to understand.Great to see a lot of gentle introduction here.



Jason Brownlee May 7, 2019 at 2:30 pm #

REPLY ↩

Thanks, I'm happy you found it useful!



Daniel May 16, 2019 at 1:38 am #

REPLY ↩

That's a weird concept..

In my mind, every node in the NN should have a specific meaning (for example, a specific node can specify a specific line that should/n't be in the classification of a car picture). When using dropout, you eliminate this "meaning" from the nodes..

What do you think about it?



Jason Brownlee May 16, 2019 at 6:34 am #

REPLY ↩

I think the idea that nodes have "meaning" at some level of abstraction is fine, but also consider that the model has a lot of redundancy which helps with its ability to generalize.



Rafis August 19, 2019 at 11:34 am #

REPLY ↩

Last point "Use With Smaller Datasets" is incorrect. Regularization confers little reduction in generalization error. In these cases, the computational cost of using dropout and larger models may outweigh the benefit of regularization.". They say that for smaller datasets regularization worked quite well. But for larger datasets regularization doesn't work and it is better to use dropout.



Jason Brownlee August 19, 2019 at 2:02 pm #

In practice, regularization with large data offers less



Liz September 9, 2019 at 5:46 am #

Just wanted to say your articles are fantastic. It's nice
I wouldn't consider myself the smartest cookie in the jar but you

Start Machine Learning

Start Machine Learning ✕

You can master applied Machine Learning
without math or fancy degrees.
Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE

posting!



Jason Brownlee September 9, 2019 at 1:52 pm #

REPLY ↩

Thanks, I'm glad the tutorials are helpful Liz!



B September 14, 2019 at 9:43 am #

REPLY ↩

Hello, it seems to me that the line:

“The default interpretation of the dropout hyperparameter is the probability of training a given node in a layer, where 1.0 means no dropout, and 0.0 means no outputs from the layer.”

The language is confusing, since you refer to the probability of a training a node, rather than the probability of a node being “dropped”. Seems you should reverse this to make it consistent with the next section where the suggestion seems to be to add more nodes when more nodes are dropped. As written in the quote above, lower dropout rate will increase the number of nodes, but I suspect it should be the inverse where the number of nodes increases with the dropout rate (more nodes dropped, more nodes needed).

B.



Jason Brownlee September 15, 2019 at 6:14 am #

REPLY ↩

Thanks for the feedback!



spring October 20, 2019 at 8:05 pm #

REPLY ↩

Aw, this was a very good post. Taking the time and actual effort to make a good article... but what can I say... I hesitate a whole lot and don't manage to get nearly anything done.



Jason Brownlee October 21, 2019 at 6:18 am #

REPLY ↩

Thanks.

Why do you think that is exactly?



Paul April 13, 2020 at 7:41 am #

Hey Jason,

Been getting your emails for a long time, just wanted to say the inspired me to create my own website 😊 So, thank you!

All the best,

Paul

Start Machine Learning

Start Machine Learning



You can master applied Machine Learning **without math or fancy degrees.**
Find out how in this *free* and *practical* course.

Email Address

START MY EMAIL COURSE



Jason Brownlee April 13, 2020 at 1:50 pm #

REPLY ↩

Thanks Paul!



Vengal May 18, 2020 at 7:22 pm #

REPLY ↩

It is mentioned in this blog “Dropout may be implemented on any or all hidden layers in the network as well as the visible or input layer. It is not used on the output layer.”

It seems that comment is incorrect. When drop-out is used for preventing overfitting, it is accurate that input and/or hidden nodes are removed with certain probability.

When dropconnect (a variant of dropout) is used for preventing overfitting, weights (instead of hidden/input nodes) are dropped with certain probability. So, there is always a certain probability that an output node will get removed during dropconnect between the hidden and output layers.

Thus, hidden as well as input/nodes can be removed probabilistically for preventing overfitting.



Jason Brownlee May 19, 2020 at 6:00 am #

REPLY ↩

Sure, you’re talking about dropconnect. Here we’re talking about dropout.



abq June 4, 2020 at 1:01 pm #

REPLY ↩

Jason, thanks a lot for the great post!

Is the final model an ensemble of models with different network structures or just a deterministic model whose structure corresponds to the best model found during the training process?



Jason Brownlee June 4, 2020 at 1:27 pm #

REPLY ↩

No. There is only one model, the ensemble is a model

Start Machine Learning



abq June 5, 2020 at 3:43 am #

REPLY ↩

Thanks!



Jason Brownlee June 5, 2020 at 8:21 am #

You’re welcome.

Start Machine Learning ×

You can master applied Machine Learning **without math or fancy degrees.** Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

Ricardo January 19, 2021 at 7:36 pm #



Thanks for this gentle introduction.



Jason Brownlee January 20, 2021 at 5:39 am #

REPLY ↩

You're welcome.



hamunak January 25, 2021 at 10:43 am #

REPLY ↩

This is an amazing summary. I admire the way you use sources instead of common dodgy claiming. Thank you.



Jason Brownlee January 25, 2021 at 10:43 am #

REPLY ↩

Thanks!



Eli February 5, 2021 at 11:26 pm #

REPLY ↩

Great, as always.
Thank you so much.



Jason Brownlee February 6, 2021 at 5:50 am #

REPLY ↩

You're welcome!

Leave a Reply

Name (required)

Email (will not be published) (required)

Website

Start Machine Learning

Start Machine Learning ×

You can master applied Machine Learning
without math or fancy degrees.
Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

SUBMIT COMMENT



Welcome!

I'm *Jason Brownlee* PhD

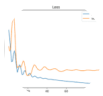
and I **help developers** get results with **machine learning**.

[Read more](#)

Never miss a tutorial:



Picked for you:



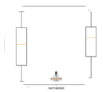
[How to use Learning Curves to Diagnose Machine Learning Model Performance](#)



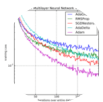
[Stacking Ensemble for Deep Learning Neural Networks in Python](#)



[How To Improve Deep Learning Performance](#)



[How to use Data Scaling Improve Deep Learning Model Stability and Performance](#)



[Gentle Introduction to the Adam Optimization Algorithm for Deep Learning](#)

Loving the Tutorial? [Start Machine Learning](#)

The [Better Deep Learning](#) EBook is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE

Start Machine Learning



You can master applied Machine Learning **without math or fancy degrees**. Find out how in this *free* and *practical* course.

START MY EMAIL COURSE

© 2021 Machine Learning Mastery Pty. Ltd. All Rights Reserved.

[LinkedIn](#) | [Twitter](#) | [Facebook](#) | [Newsletter](#) | [RSS](#)

[Privacy](#) | [Disclaimer](#) | [Terms](#) | [Contact](#) | [Sitemap](#) | [Search](#)